**1) What's the difference between HTTP and HTTPS?**

**Short summary:** HTTP is plain text; HTTPS is HTTP over TLS (encrypted, authenticated, integrity-protected). Use HTTPS for basically everything on the public web.

**What changes under the hood**

- **Encryption & integrity:** HTTPS wraps HTTP in TLS, so bytes on the wire are encrypted and tamper-protected (prevents eavesdropping and MITM).
- **Authentication:** TLS certificates let browsers verify the server's identity (certificate chain, CAs).
- **Default ports:** HTTP → port 80; HTTPS → port 443 (by convention).
- **Modern features:** Many browser/security features require HTTPS (Service Workers, HTTP/2/3 benefits, some secure cookies, geolocation APIs, etc.). HSTS tells browsers to insist on HTTPS for a host.

**Practical consequences / attacks prevented**

- Plain HTTP: attacker on same network can read and modify requests/responses (credentials, session cookies, content).
- HTTPS: prevents eavesdropping and most network-level tampering; it doesn't stop phishing or an attacker that controls the server.

**Trade-offs / operational notes**

- TLS handshake adds CPU + a (small) latency cost, but modern TLS (session tickets, TLS 1.3) and hardware/accelerators make this negligible; benefits >> cost.
- TLS termination often happens in proxies/load-balancers (so you must secure the internal network or encrypt internal hops if needed).
- Always serve everything over HTTPS (mixed content and cookies over HTTP are problem areas).

---

**2) URL segments vs fragments — what are they? (real URL example)**

Take this realistic URL

https://user:pwd@www.example.com:8080/shop/products/42/details?color=red&ref=google#reviews

Breakdown (components per RFC/URI standard):

- **Scheme:** https — protocol.
- **Userinfo (rare):** user:pwd — credentials in the URL (deprecated/ discouraged for security).
- **Authority / Host:** www.example.com and **port** 8080.

- **Path:** /shop/products/42/details — hierarchical path made of **segments**. Here the *path segments* are ["shop", "products", "42", "details"]. Route systems often map segments to route parameters (e.g. products/{id}).
- **Query string:** ?color=red&ref=google — key/value pairs (server receives this and can act on it). Query strings are sent to server.
- **Fragment:** #reviews — client-side anchor/fragment identifier. **Important:** the fragment is **not sent to the server**; it is processed by the client/browser (e.g. scroll to id="reviews" or used by client-side JS routing).

**Why the differences matter (practical effects)**

- **Fragments** never reach the server — useful for client-side navigation, in-page anchors, SPA deep links, or highlighting text, but the server can't depend on them.
- **Path segments** are part of the resource address and used by routing and caching rules; changing them generally means a different resource.
- **Query** is commonly used for filtering, pagination, or optional parameters; it's used in cache keys and can affect cacheability and CDN behavior.

**Percent-encoding / reserved characters**: spaces and special chars must be percent-encoded in segments and query values. (See RFC and browser URL APIs.)

---

**3) What are the Builder pattern and Dependency Injection (DI)? (real-life examples + when to use which)**

They solve **different** problems:

- **Builder pattern** — creational design pattern. It encapsulates complex object construction so callers can build objects step-by-step or via a fluent API. Use when creation has many optional pieces or many combinations.
- **Dependency Injection (DI)** — architectural / composition technique: supply an object's dependencies from the outside (container or composition root) rather than creating them inside. Use to decouple code, improve testability, and centralize composition. Microsoft Learn

**Real-life analogies**

- **Builder:** ordering a custom laptop — you choose CPU, RAM, disk, GPU, OS options step by step; at the end you get the assembled laptop.
- **DI:** a coffee shop where a Barista (consumer) gets a CoffeeMachine provided by the shop owner (container). The Barista doesn't build the machine; different shops may inject different machines (drip, espresso), and the Barista logic stays the same.

**Concrete C# examples**

**Builder (C# simplified):**

```
// product
public class Pizza { public bool Cheese; public bool Pepperoni; public int Size; }

// builder (fluent)
public class PizzaBuilder {
  private Pizza p = new Pizza();
  public PizzaBuilder WithCheese(){ p.Cheese = true; return this; }
  public PizzaBuilder WithPepperoni(){ p.Pepperoni = true; return this; }
  public PizzaBuilder SizeInches(int s){ p.Size = s; return this; }
  public Pizza Build(){ return p; }
}

// usage
var pizza = new PizzaBuilder()
        .WithCheese()
        .WithPepperoni()
        .SizeInches(12)
        .Build();
```

**Dependency Injection (ASP.NET Core):**

```
public interface IEmailSender { Task Send(string to, string body); }
public class SmtpEmailSender : IEmailSender { /* ... */ }

// in Program.cs / Startup
services.AddScoped<IEmailSender, SmtpEmailSender>();

// consumer (controller/service)
public class OrderService {
  private readonly IEmailSender _email;
```

```
  public OrderService(IEmailSender email) { _email = email; } // constructor injection
  public Task Notify(...) => _email.Send(...);
}
```

ASP.NET Core DI docs / guidance: Microsoft docs.

**When to use each**

- Use **Builder** when constructing a *single complex object* where the configuration process is involved and you want to hide that complexity (optional parts, many combinations).
- Use **DI** when you want to *wire up dependencies* across your application, enabling modular design and unit testing (replace implementations with mocks/fakes easily).

They can be used together: a DI container can inject a CarBuilder factory into a controller.

---

**4) Difference between Web Pages (Razor Pages) and MVC (ASP.NET) — + two business cases and comparison**

**Short answer:** Razor Pages = page-focused, co-located page model and view; MVC = controller-centric (separate controllers and views) and more explicit separation of concerns. Razor Pages is a simpler pattern for page-based scenarios; MVC is more flexible for larger, multi-controller apps and API-centric designs.

**Razor Pages (what it is)**

- Introduced in ASP.NET Core as a page-based approach: each .cshtml page has a PageModel with handler methods like OnGet()/OnPost().
- Good when the app is *page-focused* and you want the page logic colocated with the view.

**Example (Razor Page):**

```
// Pages/Product/Details.cshtml.cs
public class DetailsModel : PageModel {
  public Product Product { get; set; }
  public void OnGet(int id) { Product = _repo.Get(id); }
}
// view: Pages/Product/Details.cshtml binds to Model.Product
```

**MVC (what it is)**

- Controller (handles requests), Model (data & logic), View (render). Good for complex routing, many related actions, APIs, or when teams separate responsibilities.

**Example (MVC):**

```
public class ProductController : Controller {
  public IActionResult Details(int id) {
    var product = _repo.Get(id);
    if (product==null) return NotFound();
    return View(product); // Views/Product/Details.cshtml
  }
}
```

**Two business cases + recommended approach**

**Case A: Small company website + a few forms (contact, newsletter, product pages)**

- **Razor Pages** preferred: faster to scaffold, page-focused, simpler file layout, less ceremony. Ideal for smaller teams or page-centric flows.

**Case B: Large enterprise SaaS with modular teams, REST APIs, complex workflows, and lots of reusable controllers/views**

- **MVC** preferred: clearer separation, easier to group controllers, share view components, build API endpoints alongside controllers. Better for testability at scale and for apps that mix many concerns.

**Comparison matrix (practical)**

- **Structure:** Razor Pages = co-located page + model; MVC = controllers + views separated.
- **Best for:** Razor Pages = page-centric features; MVC = large multi-controller apps + APIs.
- **Learning curve:** Razor Pages simpler for newcomers; MVC more flexible but more boilerplate.
- **Team split:** MVC often better when backend and frontend teams strictly separate duties; Razor Pages simplifies small teams.

---

**5) What is the Content-Type in a response message, where and why do we use it?**

**Definition:** Content-Type is an HTTP **representation header** that indicates the media (MIME) type of the response body (for example text/html; charset=utf-8 or application/json). Clients use it to parse/render the response correctly.

**Common examples**

- text/html; charset=utf-8 — HTML document (browser renders it).
- application/json; charset=utf-8 — JSON API response.

- image/png — PNG image.
- application/octet-stream + Content-Disposition: attachment; filename=... — file download.

**Where used**

- **Server responses:** server sets Content-Type so the client knows how to handle the body.
- **Client requests:** Content-Type in POST/PUT tells server what format the request body is (e.g., application/json or multipart/form-data).
- **Content negotiation:** clients supply Accept header; server picks the best representation and sets Content-Type.

**Security and correctness**

- Browsers sometimes "MIME-sniff"; to prevent sniffing attacks set X-Content-Type-Options: nosniff. If you serve JSON, set Content-Type: application/json and nosniff.

**Practical snippet (ASP.NET Core)**

```
// Return JSON

return new JsonResult(data); // framework sets Content-Type application/json


// Raw response

context.Response.ContentType = "text/csv; charset=utf-8";

await context.Response.WriteAsync(csvText);
```

---

**6) Minification, bundling (web bundle), Webpack, and lazy loading — what they are and how they improve network performance**

This is an important topic — several related techniques are often used together to reduce load time and bandwidth.

**Minification**

**What:** Remove whitespace, comments, and (where safe) shorten variable names etc., to reduce file size without changing behavior. This reduces bytes transferred.

**Why it helps:** smaller files = faster transfer, lower parse time (marginal), lower bandwidth cost.

**Bundling / Module bundlers**

**What "bundling" generally means:** combining many small files (modules) into one or a few files to reduce number of HTTP requests and manage dependencies. Tools that do this are called module bundlers (e.g., Webpack).

**Webpack:** a widely used module bundler that builds a dependency graph from entry points and outputs bundles. It supports loaders (transforms), plugins, tree shaking (remove unused exports), code splitting, etc.

**Web Bundles (the WBN / Web Packaging concept)**

**What a Web Bundle is:** a *file format* that packages multiple HTTP resources into a single file (.wbn) so a site (or isolated web app) can be shipped or loaded as one bundle; it's part of the Web Packaging proposal. This is different from what people sometimes mean casually by "bundle" (i.e., concatenated JS/CSS) — Web Bundles are a standardized archive format for web resources. Use cases include distribution of offline-capable apps or signed bundles.

**Lazy loading / Code splitting**

**What:** split your code into chunks and load non-critical chunks only when needed (e.g., on route change, on user interaction). This reduces the initial payload and speeds up first meaningful paint. Examples: dynamic import() in JS, route-based splitting.

**Simple dynamic import example (React / modern JS):**

```
// React lazy + Suspense

const HeavyWidget = React.lazy(() => import('./HeavyWidget')); // creates separate chunk

// Browser downloads chunk only when HeavyWidget is rendered
```

**How these improve network performance (concrete mechanisms)**

1. **Reduce bytes transferred** — minification + tree shaking remove unused code.
2. **Reduce round trips / request overhead** — bundling reduces number of requests (helpful for high-latency networks). But note: with HTTP/2 multiplexing, the cost of many small requests is lower — so strategy must consider protocol.
3. **Improve Time to Interactive / FCP** — lazy loading pushes heavy, noncritical code later so the initial page becomes interactive faster.
4. **Better caching** — split vendor code and app code; only change app bundle hash when app code changes, allowing long cache lifetimes for vendor files.
5. **Network-optimized formats** — using binary bundles or compressed payloads (Brotli/Gzip) reduces bytes further.

**Trade-offs & pitfalls**

- **Too large single bundle**: caching becomes brittle (any small change invalidates whole bundle) and parse/compile cost is higher.
- **Too many tiny chunks**: request overhead and scheduling delays can hurt, especially on HTTP/1.1; HTTP/2/3 reduces per-request cost, so chunking strategy should be tuned. Snipcart

- **Complexity:** build config (webpack) and chunking strategy complexity can increase maintenance overhead.
- **Web Bundles spec caveats:** it's an evolving standard and has specialized use cases; don't confuse it with typical JS/CSS bundling.